

Системное администрирование

Итоговая работа

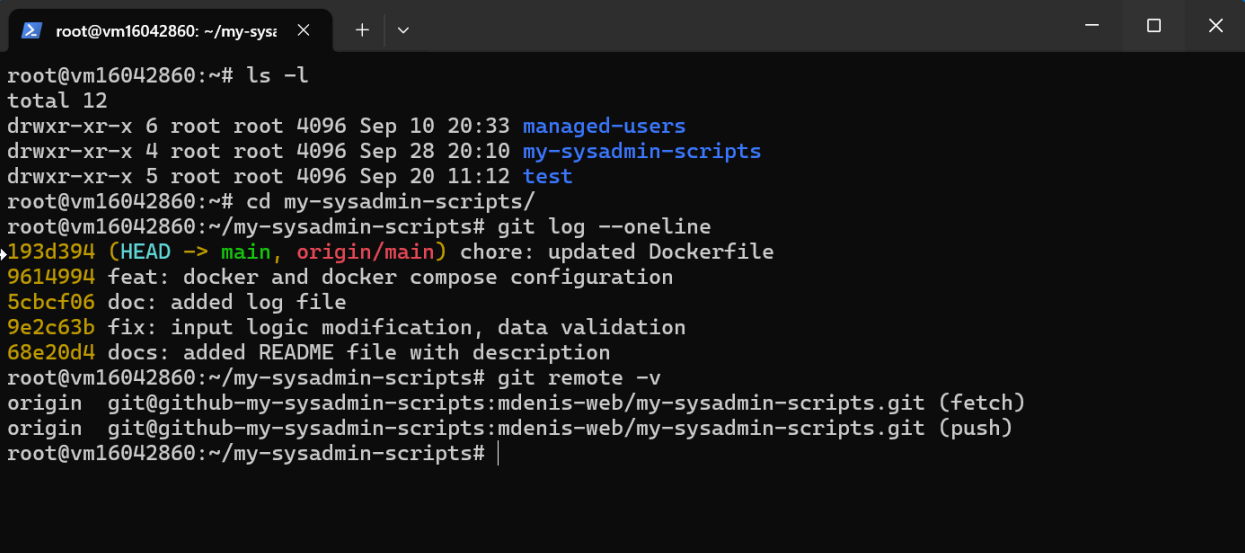
Студент: МIRONENKO Денис Николаевич
вариант: «Продвинутый»
выполнение на реальном VPS сервере

ссылка на репозиторий: <https://github.com/mdenis-web/my-sysadmin-scripts>

Часть 1. Сквозной прогон на своей рабочей машине

Пункт 1 — репозиторий, история, README

```
```bash
cd ~/my-sysadmin-scripts
git log --oneline
git remote -v
```
```

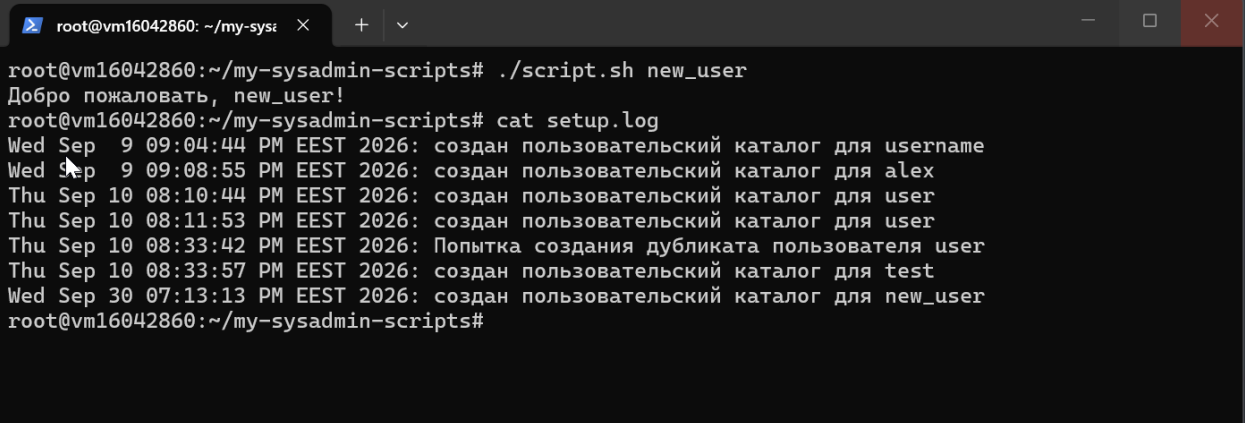
A terminal window with a dark background. The title bar shows 'root@vm16042860: ~/my-sysadmin-scripts'. The terminal output shows the following commands and results:
1. `ls -l`: Lists files in the current directory. Output shows three files: `managed-users`, `my-sysadmin-scripts`, and `test`.
2. `cd my-sysadmin-scripts/`: Changes the current directory to `my-sysadmin-scripts`.
3. `git log --oneline`: Shows the commit history. The output lists four commits with their hashes and messages: `193d394` (HEAD -> main, origin/main) chore: updated Dockerfile, `9614994` feat: docker and docker compose configuration, `5cbcf06` doc: added log file, `9e2c63b` fix: input logic modification, data validation, and `68e20d4` docs: added README file with description.
4. `git remote -v`: Shows the remote repository configuration. It lists 'origin' as `git@github-my-sysadmin-scripts:mdenis-web/my-sysadmin-scripts.git` for both fetch and push operations.

```
root@vm16042860:~# ls -l
total 12
drwxr-xr-x 6 root root 4096 Sep 10 20:33 managed-users
drwxr-xr-x 4 root root 4096 Sep 28 20:10 my-sysadmin-scripts
drwxr-xr-x 5 root root 4096 Sep 20 11:12 test
root@vm16042860:~# cd my-sysadmin-scripts/
root@vm16042860:~/my-sysadmin-scripts# git log --oneline
193d394 (HEAD -> main, origin/main) chore: updated Dockerfile
9614994 feat: docker and docker compose configuration
5cbcf06 doc: added log file
9e2c63b fix: input logic modification, data validation
68e20d4 docs: added README file with description
root@vm16042860:~/my-sysadmin-scripts# git remote -v
origin  git@github-my-sysadmin-scripts:mdenis-web/my-sysadmin-scripts.git (fetch)
origin  git@github-my-sysadmin-scripts:mdenis-web/my-sysadmin-scripts.git (push)
root@vm16042860:~/my-sysadmin-scripts#
```

- ✓ репозиторий существует,
- ✓ история — не менее трёх осмысленных коммитов,
- ✓ имеется адрес на GitHub.

Пункт 2 — скрипт автоматизации

```
```bash
./script.sh new_user
cat setup.log
```
```



```
root@vm16042860: ~/my-sysadmin-scripts
root@vm16042860:~/my-sysadmin-scripts# ./script.sh new_user
Добро пожаловать, new_user!
root@vm16042860:~/my-sysadmin-scripts# cat setup.log
Wed Sep  9 09:04:44 PM EEST 2026: создан пользовательский каталог для username
Wed Sep  9 09:08:55 PM EEST 2026: создан пользовательский каталог для alex
Thu Sep 10 08:10:44 PM EEST 2026: создан пользовательский каталог для user
Thu Sep 10 08:11:53 PM EEST 2026: создан пользовательский каталог для user
Thu Sep 10 08:33:42 PM EEST 2026: Попытка создания дубликата пользователя user
Thu Sep 10 08:33:57 PM EEST 2026: создан пользовательский каталог для test
Wed Sep 30 07:13:13 PM EEST 2026: создан пользовательский каталог для new_user
root@vm16042860:~/my-sysadmin-scripts#
```

- ✓ скрипт решает исходную задачу без обращения к контейнеру

Пункт 3 — Dockerfile и контейнер

```
```bash
docker build -t my-script .
docker ps -a
docker images
```
```

```
root@vm16042860: ~/my-sysadmin-scripts# docker build -t my-script .
docker ps -a
docker images
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 108.5kB
Step 1/6 : FROM ubuntu:22.04
----> b8b6ee6aa931
Step 2/6 : RUN apt-get update && apt-get install -y --no-install-recommends python3 && rm -rf /var/lib/apt/lists/*
----> Using cache
----> 5834ee55fb8c
Step 3/6 : COPY script.sh /usr/local/bin/script.sh
----> Using cache
----> 901f3ef37a03
Step 4/6 : RUN chmod +x /usr/local/bin/script.sh
----> Using cache
----> 63e4a1798a1c
Step 5/6 : WORKDIR /root
----> Using cache
----> 42ffed75f42d
Step 6/6 : CMD ["/bin/bash", "-c", "/usr/local/bin/script.sh demo-user && exec python3 -m http.server 8080"]
----> Using cache
----> aa49905e7ddc
Successfully built aa49905e7ddc
Successfully tagged my-script:latest
```

| CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS | PORTS |
|--------------|-----------|--------------------------|--------------|-------------|---|
| e36deb8a0c88 | my-script | "/bin/bash -c '/usr/..." | 46 hours ago | Up 46 hours | 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp |
| my-app | | | | | |

Info → U In Use

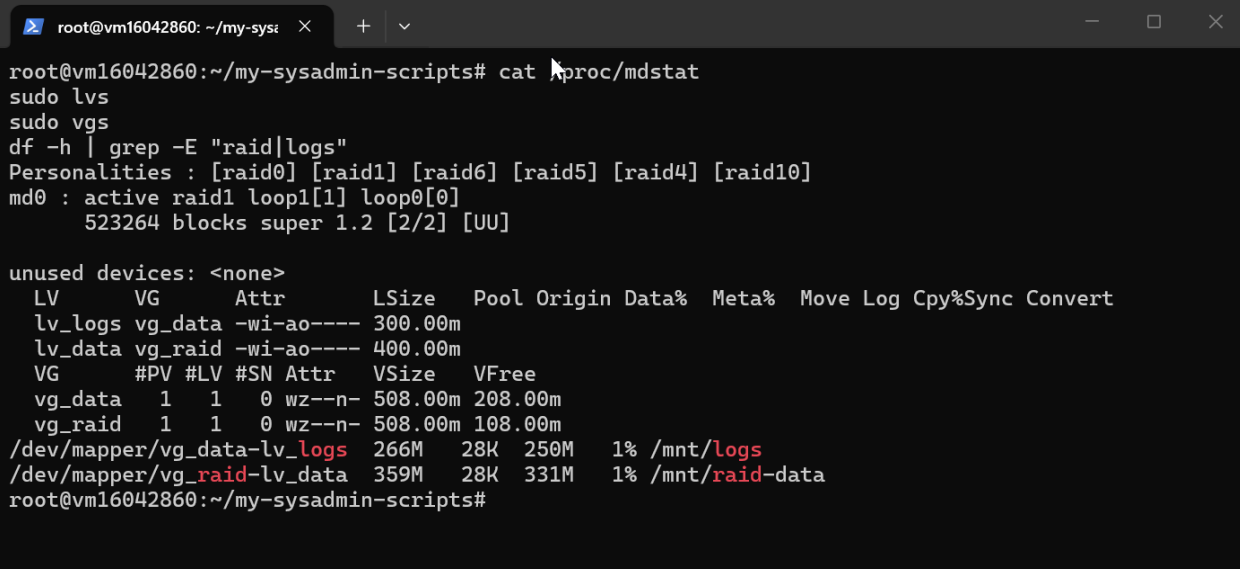
| IMAGE | ID | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------------|--------------|------------|--------------|-------|
| alpine:latest | 5b02b42e375f | 13MB | 3.94MB | |
| env-test:latest | de3ec6e26c19 | 12.9MB | 3.85MB | |
| hello-world:latest | 5e2309035332 | 25.9kB | 9.49kB | |
| my-script:latest | aa49905e7ddc | 161MB | 40.1MB | U |
| my-script:v1 | c27f640362b5 | 188MB | 46.2MB | |
| myapp:v1 | 0264a80d69eb | 181MB | 44.9MB | |
| nginx:alpine | 62ff2089abf5 | 94.4MB | 27.2MB | |
| node:18-alpine | 8d6421d663b4 | 181MB | 45.3MB | |
| python:3.12-slim | 78387bc3881b | 190MB | 48.4MB | |
| ubuntu:22.04 | b8b6ee6aa931 | 119MB | 31.7MB | |

```
root@vm16042860:~/my-sysadmin-scripts#
```

- ✓ собран Docker-образ из Dockerfile
- ✓ показан список всех контейнеров (включая остановленные)
- ✓ показан список локальных образов

Пункт 4 — RAID/LVM

```
``bash
cat /proc/mdstat
sudo lvs
sudo vgs
df -h | grep -E "raid|logs"
``
```



The screenshot shows a terminal window with the following output:

```
root@vm16042860: ~/my-sysadmin-scripts# cat /proc/mdstat
Personalities : [raid0] [raid1] [raid6] [raid5] [raid4] [raid10]
md0 : active raid1 loop1[1] loop0[0]
      523264 blocks super 1.2 [2/2] [UU]

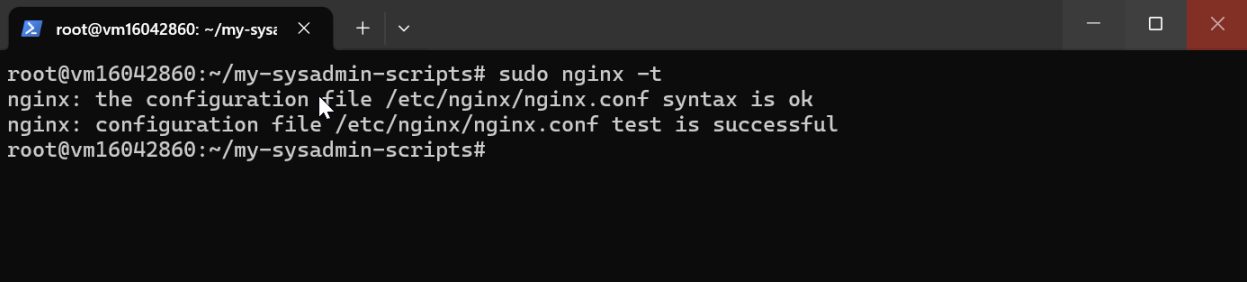
unused devices: <none>

LV      VG      Attr      LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
lv_logs vg_data -wi-ao---- 300.00m
lv_data vg_raid -wi-ao---- 400.00m
VG      #PV #LV #SN Attr   VSize   VFree
vg_data  1  1  0 wz--n- 508.00m 208.00m
vg_raid  1  1  0 wz--n- 508.00m 108.00m
/dev/mapper/vg_data-lv_logs 266M   28K   250M    1% /mnt/logs
/dev/mapper/vg_raid-lv_data 359M   28K   331M    1% /mnt/raid-data
root@vm16042860:~/my-sysadmin-scripts#
```

- ✓ RAID1 здоров, деградации нет. Он собран из loop-устройств
- ✓ оба тома активны, доступны на запись и используются (смонтированы)
- ✓ обе группы томов здоровы, есть свободное место для расширения LV.
- ✓ занято по 1% файловой системы

Пункт 5 — Nginx как обратный прокси

```
```bash  
sudo nginx -t
```
```

A terminal window with a dark background. The title bar shows 'root@vm16042860: ~/my-sys:'. The terminal text shows the command 'sudo nginx -t' being executed, followed by two lines of output: 'nginx: the configuration file /etc/nginx/nginx.conf syntax is ok' and 'nginx: configuration file /etc/nginx/nginx.conf test is successful'. The prompt returns to 'root@vm16042860:~/my-sysadmin-scripts#'.

```
root@vm16042860:~/my-sysadmin-scripts# sudo nginx -t  
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginx.conf test is successful  
root@vm16042860:~/my-sysadmin-scripts#
```

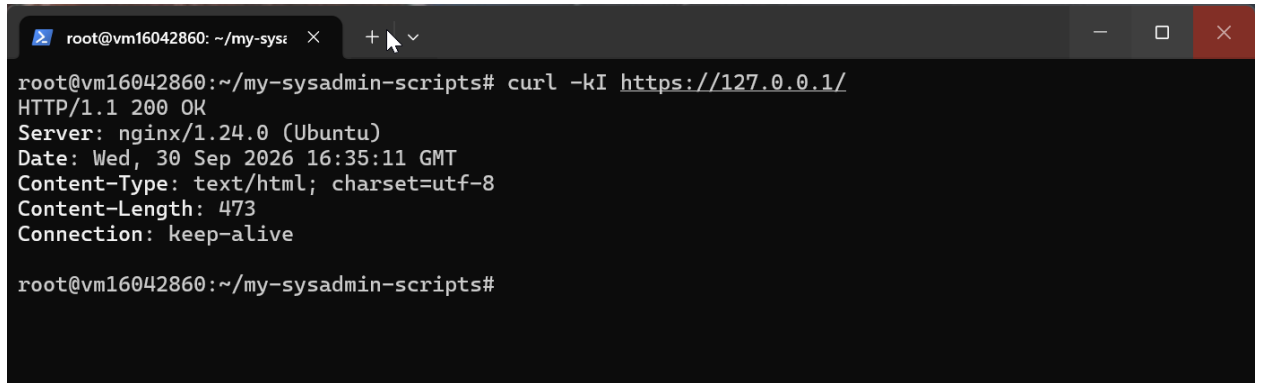
- ✓ syntax is ok
- ✓ test is successful

Пункт 6 — HTTPS

```
```bash
```

```
curl -kI https://127.0.0.1/
```

```
```
```

A terminal window with a dark background. The title bar shows 'root@vm16042860: ~/my-sysadmin-scripts' and standard window controls. The terminal text shows a successful curl command output.

```
root@vm16042860:~/my-sysadmin-scripts# curl -kI https://127.0.0.1/
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 30 Sep 2026 16:35:11 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 473
Connection: keep-alive

root@vm16042860:~/my-sysadmin-scripts#
```

✓ код ответа 200, сервис работает

Пункт 7 — systemd-служба

```
```bash
```

```
sudo systemctl status my-app
```

```
```
```

```
root@vm16042860: ~/my-sysadmin-scripts# sudo systemctl status my-app
● my-app.service - my-app service
   Loaded: loaded (/etc/systemd/system/my-app.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-28 20:48:05 EEST; 1 day 22h ago
     Main PID: 178839 (docker)
        Tasks: 9 (limit: 2263)
      Memory: 8.8M (peak: 9.3M)
         CPU: 5.235s
    CGroup: /system.slice/my-app.service
            └─178839 docker start -a my-app

Sep 30 18:16:04 vm16042860.example.com docker[178839]: 205.237.104.18 - - [30/Sep/2026 15:16:04] "GET / HTTP/1.1" 200 1524
Sep 30 18:34:34 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 15:34:34] "GET / HTTP/1.1" 200 1524
Sep 30 18:39:20 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 15:39:20] "GET / HTTP/1.1" 200 1524
Sep 30 18:54:21 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 15:54:21] "GET / HTTP/1.1" 200 1524
Sep 30 18:54:21 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 15:54:21] "GET / HTTP/1.1" 200 1524
Sep 30 19:19:30 vm16042860.example.com docker[178839]: 44.211.148.114 - - [30/Sep/2026 16:19:30] "GET / HTTP/1.1" 200 1524
Sep 30 19:19:48 vm16042860.example.com docker[178839]: 220.48.114.19 - - [30/Sep/2026 16:19:48] "GET / HTTP/1.1" 200 1524
Sep 30 19:22:39 vm16042860.example.com docker[178839]: 54.176.45.57 - - [30/Sep/2026 16:22:39] "GET / HTTP/1.1" 200 1524
Sep 30 19:30:27 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 16:30:27] "GET / HTTP/1.1" 200 1524
Sep 30 19:35:11 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 16:35:11] "GET / HTTP/1.1" 200 1524
lines 1-20/20 (END)
```

- ✓ active
- ✓ service enabled

Пункт 8 — логи

```
```bash
```

```
sudo journalctl -u my-app -n 10 --no-pager
```

```
sudo tail -n 10 /var/log/nginx/access.log
```

```
```
```

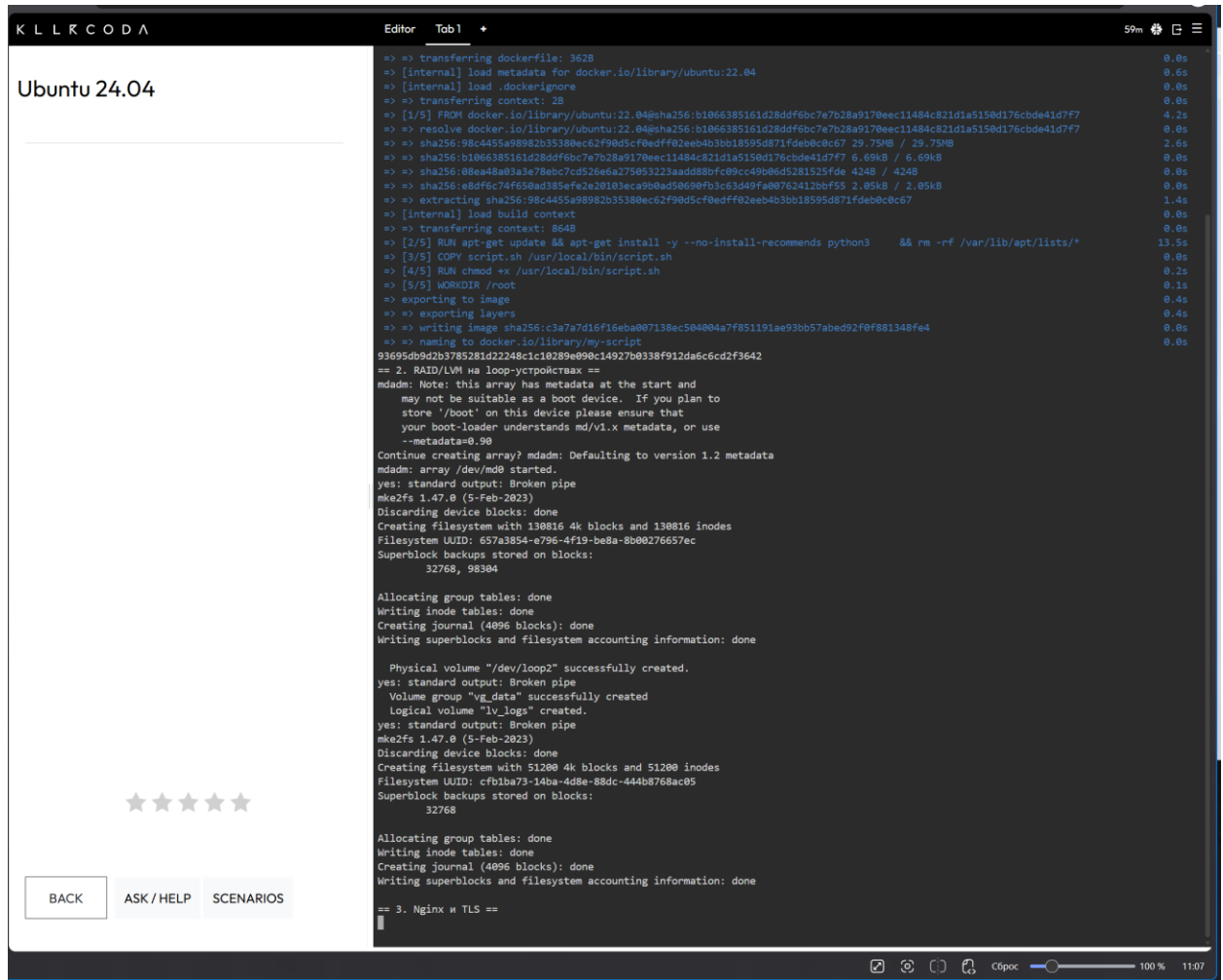
```
root@vm16042860: ~/my-sysadmin-scripts# sudo journalctl -u my-app -n 10 --no-pager
sudo tail -n 10 /var/log/nginx/access.log
Sep 30 19:19:30 vm16042860.example.com docker[178839]: 44.211.148.114 - - [30/Sep/2026 16:19:30] "GET / HTTP/1.1" 200 -
Sep 30 19:19:48 vm16042860.example.com docker[178839]: 220.48.114.19 - - [30/Sep/2026 16:19:48] "GET / HTTP/1.0" 200 -
Sep 30 19:22:39 vm16042860.example.com docker[178839]: 54.176.45.57 - - [30/Sep/2026 16:22:39] "GET / HTTP/1.1" 200 -
Sep 30 19:30:27 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 16:30:27] "GET / HTTP/1.0" 200 -
Sep 30 19:35:11 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 16:35:11] "HEAD / HTTP/1.0" 200 -
Sep 30 19:35:56 vm16042860.example.com docker[178839]: 172.17.0.1 - - [30/Sep/2026 16:35:56] "GET / HTTP/1.0" 200 -
Sep 30 19:37:09 vm16042860.example.com docker[178839]: 20.65.145.128 - - [30/Sep/2026 16:37:09] code 400, message Bad request version ('À\x14À')
Sep 30 19:37:09 vm16042860.example.com docker[178839]: 20.65.145.128 - - [30/Sep/2026 16:37:09] "\x16\x03\x01\x00i\x01\x00\x00\x03\x033.gUÆtGC\x93#8\x1a,Z¥\x80\x7fä;bn\x1a,\x12\x88,İ>\x9cQ"\x83 0#\x99|\x1fæ\x08& ^<"¥\x8b;ÿË\x18^ÇBS\x90Y\x1e¥%ûßÆÛ·\x00&I"İ@À/À0À+À,À\x13À\x09À\x14À" 400 -
Sep 30 19:37:09 vm16042860.example.com docker[178839]: 20.65.145.128 - - [30/Sep/2026 16:37:09] code 400, message Bad request syntax ('MGLNDD_45.136.126.128_8080')
Sep 30 19:37:09 vm16042860.example.com docker[178839]: 20.65.145.128 - - [30/Sep/2026 16:37:09] "MGLNDD_45.136.126.128_8080" 400 -
45.198.224.125 - - [30/Sep/2026:18:44:27 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:71.0) Gecko/20100101 Firefox/71.0"
139.84.241.211 - - [30/Sep/2026:18:54:21 +0300] "GET /admin/config.php HTTP/1.0" 404 469 "-" "xfal"
205.237.104.18 - - [30/Sep/2026:19:13:05 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.85 Safari/537.36 Edg/90.0.818.46"
54.167.71.13 - - [30/Sep/2026:19:18:26 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36"
91.92.40.236 - - [30/Sep/2026:19:18:49 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.85 Safari/537.36 Edg/90.0.818.46"
185.12.59.118 - - [30/Sep/2026:19:30:27 +0300] "GET / HTTP/1.1" 200 300 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0"
45.79.8.221 - - [30/Sep/2026:19:34:30 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 zgrab/0.x"
127.0.0.1 - - [30/Sep/2026:19:35:11 +0300] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.5.0"
45.33.80.243 - - [30/Sep/2026:19:35:56 +0300] "GET / HTTP/1.1" 301 178 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 13_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/108.0.0.0 Safari/537.36"
45.33.80.243 - - [30/Sep/2026:19:35:56 +0300] "GET / HTTP/1.1" 200 300 "http://45.136.126.128/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 13_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/108.0.0.0 Safari/537.36"
root@vm16042860:~/my-sysadmin-scripts#
```

- ✓ показаны последние 10 строк логов systemd-юнита my-app
- ✓ показаны последние 10 строк лога доступа Nginx

ИТОГО: Все восемь групп команд выдают положительный результат, службы запущены и работают на удаленном сервере.

Bootstrap.sh

Bootstrap.sh собран для варианта В и проверен на свежей машине killercoda.com



```
=> => transferring dockerfile: 362B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/ubuntu:22.04@sha256:b1066385161d28ddfbc7e7b28a9170eeca11484c821d1a5150d176cbde41d7f7
=> => resolve docker.io/library/ubuntu:22.04@sha256:b1066385161d28ddfbc7e7b28a9170eeca11484c821d1a5150d176cbde41d7f7
=> => sha256:98c4455a98982b35380ec62f90d5cf0edff02eeb4b3bb18595d871fdeb0c0c67 29.75MB / 29.75MB
=> => sha256:b1066385161d28ddfbc7e7b28a9170eeca11484c821d1a5150d176cbde41d7f7 6.69kB / 6.69kB
=> => sha256:08ea48a83a3e78ebc7cd5266a275053223aadd88bfc09cc49b06d5281525fde 424B / 424B
=> => sha256:e8df6c74f650ad385efe2e20103eca9b0ad50690fb3c3d49fa0762412b0f55 2.05kB / 2.05kB
=> => extracting sha256:98c4455a98982b35380ec62f90d5cf0edff02eeb4b3bb18595d871fdeb0c0c67
=> [internal] load build context
=> transferring context: 864B
=> [2/5] RUN apt-get update && apt-get install -y --no-install-recommends python3 && rm -rf /var/lib/apt/lists/*
=> [3/5] COPY script.sh /usr/local/bin/script.sh
=> [4/5] RUN chmod +x /usr/local/bin/script.sh
=> [5/5] WORKDIR /root
=> exporting to image
=> exporting layers
=> writing image sha256:c3a7a7d16f16eba007138ec50400a7f851191ae93bb57abed92f0f881348f4
=> naming to docker.io/library/my-script
93695d0d2b3785281d2248c1c1030e090c14927b0338f912da6c6cd2f3642
== 2. RAID/LVM на loop-ycrcpocraax ==
mdadm: Note: this array has metadata at the start and
may not be suitable as a boot device. If you plan to
store '/boot' on this device please ensure that
your boot-loader understands md/v1.x metadata, or use
--metadata=0.90
Continue creating array? mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
yes: standard output: Broken pipe
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 130816 4k blocks and 130816 inodes
Filesystem UUID: 657a3854-e796-4f19-b88a-8b00276657ec
Superblock backups stored on blocks:
32768, 98304

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

Physical volume "/dev/loop2" successfully created.
yes: standard output: Broken pipe
Volume group "vg_data" successfully created
Logical volume "lv_logs" created.
yes: standard output: Broken pipe
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 51200 4k blocks and 51200 inodes
Filesystem UUID: cfb1ba73-14ba-4d8e-88dc-444b8768ac05
Superblock backups stored on blocks:
32768

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

== 3. Nginx и TLS ==
```

```
== 5. Самопроверка ==
Ожидание готовности сервиса
HTTP/1.1 200 OK
Personalities : [raid0] [raid1] [raid6] [raid5] [raid4] [raid10]
md0 : active raid1 loop1[1] loop0[0]
523264 blocks super 1.2 [2/2] [UU]

unused devices: <none>
/dev/md0 463M 24K 427M 1% /mnt/raid
/dev/mapper/vg_data-lv_logs 172M 24K 158M 1% /mnt/logs
● my-app.service - my-app service
Loaded: loaded (/etc/systemd/system/my-app.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-10-03 08:06:46 UTC; 5s ago
Main PID: 3579 (docker)
Tasks: 7 (limit: 2237)
Memory: 7.3M (peak: 7.6M)
CPU: 18ms
CGroup: /system.slice/my-app.service
└─3579 docker start -a my-app

Oct 03 08:06:46 ubuntu systemd[1]: my-app.service: Scheduled restart job, restart counter is at 1.
Oct 03 08:06:46 ubuntu systemd[1]: Started my-app.service - my-app service.
Oct 03 08:06:47 ubuntu docker[3579]: Добро пожаловать, demo-user!
Oct 03 08:06:51 ubuntu docker[3579]: 172.17.0.1 - - [03/Oct/2026 08:06:51] "GET / HTTP/1.0" 200 -
Oct 03 08:06:51 ubuntu docker[3579]: 172.17.0.1 - - [03/Oct/2026 08:06:51] "HEAD / HTTP/1.0" 200 -
bootstrap.sh: готово
```

- | | |
|--------------------------------------|----|
| ✓ Самопроверка сервиса (Nginx + TLS) | OK |
| ✓ RAID-массив | OK |
| ✓ LVM (Logical Volume Manager) | OK |
| ✓ Systemd-служба | OK |

Скрипт завершился сообщением: **«bootstrap.sh: готово»**, и все проверки (RAID, LVM, Nginx, systemd) прошли успешно.

Часть 2. Архитектурные решения

1. Почему выбран именно этот вариант скрипта (А/Б/В из Д31)?

Мною выбран вариант **В** — `script.sh` принимает имя пользователя первым позиционным аргументом (`USERNAME="$1"`), а Docker образ изначально собирался с `ENTRYPOINT ["/usr/local/bin/script.sh"]`.

Причины:

- Аргумент, а не захартоженное значение позволяет запускать один и тот же образ с разными пользователями без его пересборки. Так как `ENTRYPOINT` получает всё, что идёт после имени образа в `docker run`, как позиционные аргументы. Вариант с `CMD` в `shell`-форме или с именем, зашитым внутрь `script.sh`, пришлось бы пересобирать под каждое значение.
- Этот образ мы можем использовать и в разовом `docker run`, и в `docker compose` (`command: ["${USERNAME:-guest}"]`), и в `systemd`-службе. `ENTRYPOINT` в `exec`-форме корректно работает во всех трёх: аргумент из `command` в `compose` подставляется поверх `ENTRYPOINT`, а `systemd` просто запускает уже созданный контейнер.
- Образ остаётся неизменным, а всё, что меняется между запусками, выносится наружу в аргумент. Мы можем использовать `.env` или переменную окружения. Таким образом мы отделяем образ от параметров делая его более универсальным.
- При пустом аргументе скрипт не ломается, а переходит в запасной сценарий — интерактивный запрос. Если и там пусто — явно завершается с ошибкой, а не создаёт мусор. Таким образом остается безопасным при пустом аргументе.

В окончательной версии `Dockerfile` я убрал `ENTRYPOINT`, а запуск идёт через `CMD ["/bin/bash", "-c", "...script.sh demo-user && exec python3 -m http.server 8080"]`. Это сделано из-за `shell`-формы `CMD` аргумент из `docker run/docker create` не пробрасывается в `script.sh` как `$1` и внутри контейнера всегда запускается “demo-user”. Это сделано, чтобы поднять HTTP сервер рядом со скриптом, а не сохранить параметризацию. Вариант В при этом остаётся рабочим для `script.sh`

2. Что из RAID 1 и LVM было собрано и какую задачу решает каждый слой

В лабораторной работе я собрал и **RAID 1** и **LVM**. По своей сути – это не альтернативы, а два разных слоя:

- **RAID 1** Два loop-устройства disk1.img и disk2.img объединены в /dev/md0 уровнем raid1, на нём — ext4, смонтирован в /mnt/raid.
- **LVM** Третье loop-устройство disk3.img отдано под физический том pvcreate, из него создана группа vg_data, внутри — логический том lv_logs на 200 МБ, на нём — ext4, смонтирован в /mnt/logs.

RAID 1 отвечает за отказоустойчивость за счёт дублирования записи данных. То есть в реализованной мною схеме, одни и те же данные пишутся на оба диска одновременно. При выходе из строя одного из них массив продолжает работать на втором, а после замены диска данные восстанавливаются из копии. За отказоустойчивость мы платим емкостью носителя – вместо 1ГБ получилось 512 МБ, а также сниженной скоростью записи, так как ее необходимо выполнять на двух носителях. Таким образом RAID 1 — это защита данных, а не управление объёмом.

LVM отвечает за гибкое управление объёмом дисков. Поверх физического тома строится группа томов, внутри которой логические тома можно создавать, расширять и сжимать независимо от физической разметки. К примеру, на ходу можно расширить логический том lvextend -L +100M и увеличить файловую систему командой resize2fs Том увеличится без размонтирования и без переразбивки диска. Таким образом LVM управляет логическими томами.

RAID 1 и **LVM** это слои с разной ответственностью и задачами. RAID 1 задает отказоустойчивость, а LVM гибкость системы.

3. Почему выбран именно такой способ получения сертификата?

Мною был использован самоподписанный сертификат через `openssl req -x509`. Для этого использовал команду `bash`:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/my-app.key \
-out /etc/ssl/certs/my-app.crt \
-subj "/CN=my-app.local"
```

Причины:

- Лабораторная работа и результат локальный, нет домена. В нашей работе Nginx слушает `server_name` и отвечает на API 127.0.0.1. Для такого имени Let's Encrypt не выдаст сертификат, нужен реальный домен
- `-subj` вместо интерактивного диалога. Без `-subj` `openssl req` задаёт вопросы (страна, организация, Common Name). Это необходимо для автоматического прогона скрипта `bootstrap.sh`, иначе скрипт зависнет без ответа
- `-nodes` — No DES — не шифровать приватный ключ паролем. Иначе при каждом `systemctl reload nginx` нужно было бы вводить пароль, что противоречит вообще понятию автоматического режима.
- `-days 365` и `rsa:2048` — Срок действия сертификата 365 дней с момента создания, сгенерированный алгоритмом RSA длиной 2048 бит (стандарт для самоподписанных сертификатов). Достаточно для нашей работы и принимается современными клиентами без предупреждений о слабой криптографии.
- Проверка идёт с `curl -k`. Флаг `-k` отключает проверку цепочки доверия, так как сертификат является самоподписанным. В системных корневых хранилищах его нет, это то что необходимо нашей работы.

4. Какие альтернативы рассматривались и почему от них отказались

Получения сертификата:

- Let's Encrypt. Даёт настоящий доверенный сертификат, но требует публичного домена и доступа извне для ACME-челленджа. Домена нет, челлендж не пройдёт.
- Без TLS вообще, только HTTP. Возможно, но тогда не проверяется ни редирект return 301 https://..., ни listen 443 ssl, ни проксирование по HTTPS.

По способу организации хранилища:

- Только RAID без LVM. Закрывает обязательный минимум, но не имеет гибкости управления томами (lvextend + resize2fs на ходу).
- Только LVM без RAID. Даёт гибкость, но не даёт отказоустойчивости.
- Реальные разделы/диски вместо loop-устройств. Но их нет в наличии.

По способу запуска контейнера в финале:

- docker compose как постоянная схема. Удобен, но не превращает контейнер в системную службу и не даёт автозапуска через systemctl enable.
- docker run -d без systemd. Работает до перезагрузки/рестарта Docker, автозапуска нет. Нет полноценной системной службы.
- docker compose с restart: always. Даёт перезапуск, но не даёт управления через systemctl (start/stop/status/enable) и не даёт логов в journalctl -u my-app.

5. Что не получилось с первого раза и как было исправлено

Были некоторые нюансы с установкой docker compose. Нашел и применил подходящую схему.

Отказывался запускаться скрипт script.sh, bootstrap.sh после создания. Решил добавлением прав.

В целом ничего критичного.